Two-week progress update

A cleaner Rust IDXD path, with a small hardware proof

Hongtao Zhang

Progress window: Apr 16–30, 2026

> Small claim for this update: **the Rust IDXD path is now cleaner, easier to test, and has a small real-hardware proof**. This is **not yet** a claim that IDXD speeds up Tonic end to end.

The short version

Tonic result

1. I cleaned up the Tonic evidence package.
2. The current data does not show an IDXD speedup.
3. A stronger Tonic result needs a fresh prepared-host rerun.

Rust IDXD work

1. Moved async memmove to explicit owned buffers.
2. Added a direct Tokio completion path.
3. Replaced legacy crate names with `idxd-rust` and `idxd-sys`.

Hardware proof

1. Added one shared session shape for DSA and IAX.
2. Ran DSA memmove and IAX crc64 on prepared hardware.
3. Collected a small release-mode measurement.

The update is useful because it narrows the story. We now know what is proved, what is not proved, and what should be tested next.

Work done during the two weeks

Work volume

Commits

136

non-merge commits since Apr 16

Active days

6

Apr 24, 25, 27, 28, 29, 30

Main crates

3

`idxd-rust`, `idxd-sys`, `hw-eval`

Deck claim

small

library progress plus limited hardware proof

Main phases

Apr 24–27: clean up the evidence boundary

Tonic evidence was rewritten around what the current package can actually show. Legacy package names were also consolidated.

Make the story honest and easier to follow.

Apr 27–28: build the async path

The async API now uses caller-owned buffers, and direct Tokio completion is the main path.

Make async IDXD testable and observable.

Apr 29–30: simplify and generalize

The hardware Rust code was split by responsibility, then DSA and IAX were connected through one shared session pattern.

Make the code easier to extend.

Tonic: the current result is negative or inconclusive

Do not claim a Tonic speedup yet. The current package proves that the workflow exists, but the current rows do not show IDXD beating the ordinary software path.

Part	State	Plain meaning
Software path	**works**	The ordinary Tonic workloads can be validated and packaged.
IDXD path	**gated**	The IDXD verifier is in place, but a clean live rerun needs the prepared host to pass preflight.
Claim package	**stable**	The workflow emits JSON, CSV, and markdown summaries that can be reviewed.
Current numbers	**no win**	The current rows show IDXD at about `0.003x`–`0.653x` of software throughput.

Small claim

The Tonic measurement workflow is now clearer and easier to rerun. It does not currently support a positive acceleration claim.

Next proof needed

Run the IDXD side again on a prepared host, then rebuild the comparison package from fresh live artifacts.

Rust IDXD API: make ownership explicit

Public API change

`AsyncMemmoveRequest::new(source: Bytes, destination: BytesMut)`

The caller provides both buffers. The library does the memmove and returns the destination plus a validation report.

Why this is simpler

The API no longer hides destination allocation or copy-back behavior. The caller can see exactly what memory is submitted.

Direct Tokio path

1. Each accepted operation owns its descriptor, completion record, source, and destination until it finishes.
2. Completion records, not submit acceptance alone, decide when the future resolves.
3. Backpressure, retry count, completion status, and validation phase stay visible in errors.
4. Payload bytes are not printed in diagnostics.

Software batching and alternate submit paths are still future work. They are not needed to explain the current API.

Package cleanup: one safe crate, one raw crate

Before

1. Safe code lived under `dsa-ffi`.
2. Raw bindings were named `idxd-bindings`.
3. Wrapper scripts made it hard to tell which path was current.
4. Downstream code still pointed at old names.

Now

1. `idxd-rust` is the safe Rust and Tokio-facing crate.
2. `idxd-sys` is the raw UAPI and MMIO crate.
3. Compatibility wrappers point to the new scripts.
4. A package inventory check catches old active references.

This is not a performance result. It is a cleanup result: future work has fewer names to reason about and fewer stale entrypoints to trip over.

Code quality: easier to navigate, less magic

Area	What changed	What did not change
Builders and errors	Used builders and SNAFU errors only where they made config or diagnostics clearer.	No builder was added around raw descriptors, request buffers, benchmark hot loops, or report records.
Module split	`idxd-rust`, `idxd-sys`, and `hw-eval` now have clearer owner modules and guard scripts.	Public APIs, JSON fields, verifier output, and raw hardware behavior were kept stable.
Raw boundary	`idxd-sys` now separates descriptor, portal, completion, timing, topology, and cache helpers.	Low-level facts such as OS errors, volatile status reads, and ENQCMD accepted/rejected results stay visible.

The practical benefit is maintenance: a future change should now have a clearer owner, a clearer test, and a smaller chance of duplicating lifecycle code.

Generic IDXD session: shared shape for DSA and IAX

What was added

1. `IdxdSession<Dsa>` for DSA memmove.
2. `IdxdSession<Iax>` for IAX crc64.
3. One portal owner and one config shape.
4. Separate operation code for DSA and IAX details.

What is shared

1. Reset and fill a descriptor.
2. Submit it to the work queue.
3. Watch the completion record.
4. Classify success, retry, or failure.
5. Return typed operation results.

This is intentionally small. It does not try to cover every DSA or IAX operation yet.

Small hardware proof

Evidence	Target	Device	Bytes	Iters	Mean latency	Rate
Operation proof	dsa-memmove	/dev/dsa/wq0.0	64	n/a	completed	pass
Operation proof	iax-crc64	/dev/iax/wq1.0	64	n/a	completed	crc ok
Small bench	dsa-memmove	/dev/dsa/wq0.0	4096	1000	6,837 ns	146,246 ops/s
Small bench	iax-crc64	/dev/iax/wq1.0	4096	1000	2,178 ns	459,064 ops/s

Verifier

pass

operation proof and small benchmark both passed

Build

release

measurements were collected in release mode

Failures

0 / 2000

benchmark operations completed without failed rows

This is a proof that the new path works on two representative operations. It is not a full benchmark study.

What this means

Safe to say now

1. The Rust IDXD code is in a cleaner shape.
2. Async memmove has a clearer ownership model.
3. DSA memmove and IAX crc64 both ran through the new generic session path.
4. The small release-mode benchmark produced positive metrics for both rows.

Do not say yet

1. IDXD speeds up Tonic end to end.
2. The benchmark results generalize across sizes or workloads.
3. The generic session covers the full DSA/IAX surface.
4. The code is ready for production scheduling or batching.

The small conclusion is enough: the library path is cleaner, the proof path is real, and the next Tonic claim needs a focused rerun.

Next step

1. Rerun Tonic evidence

Use a prepared host to refresh the IDXD side, then rebuild the ordinary-vs-IDXD comparison package.

Best next step for an advisor update.

2. Add one more operation

Extend the generic session only when there is a real consumer and a verifier for the new operation.

Best next step for library growth.

3. Clean verifier helpers

Extract shared launcher and artifact helpers only after one more verifier repeats the same pattern.

Best next step if scripts become painful.

Recommended next step: rerun the Tonic comparison on a prepared host, because that is the result closest to the original project question.